Projeto: API de Biblioteca

Especificação para implementar do zero, com pouca ajuda: arquitetura, tabelas, endpoints, regras de negócio e roteiro por fases.

1. Visão geral

Uma API REST para uma biblioteca: catálogo de livros, controle de exemplares físicos, empréstimos com prazo e multa, reservas e três níveis de acesso.

O que este projeto treina além da API de tarefas

Novidade	Onde aparece
Relacionamento muitos-para-muitos	Livro ↔ Autor
Relacionamentos em cadeia	Categoria → Livro → Exemplar → Empréstimo
Regras de negócio com vários passos	Emprestar, devolver, renovar, multar
Transações e concorrência	Dois atendentes emprestando a última cópia
Três papéis com hierarquia	MEMBRO, BIBLIOTECARIO, ADMIN
Rotas públicas só para leitura (GET)	Catálogo aberto, escrita protegida
Paginação e filtros combinados	Busca de livros e empréstimos
Configuração tipada	Prazo, limites e multa no `application.yml`
Tempo testável	Testar atraso sem esperar dias (injetar `Clock`)
Restrições no banco	Índices únicos parciais, `CHECK`

Arquivos que acompanham este documento

- `diagrama-classes-biblioteca.puml`: diagrama de classes em nível de projeto.
- `diagrama-er-biblioteca.puml`: tabelas e relacionamentos.

2. Atores e permissões

Os papéis são hierárquicos: **ADMIN inclui BIBLIOTECARIO, que inclui MEMBRO**.

Papel	Pode
Visitante (sem login)	Consultar o catálogo (livros, autores, categorias)
MEMBRO	Tudo do visitante, ver os próprios empréstimos, renovar, reservar e cancelar reservas
BIBLIOTECARIO	Tudo do membro, mais gerenciar o catálogo e exemplares, registrar empréstimo, devolução e pagamento de multa, ver empréstimos de todos
ADMIN	Tudo do bibliotecário, mais listar usuários, alterar papéis e ativar ou desativar contas

Quem faz o empréstimo? O bibliotecário registra o empréstimo para um membro. O membro não se autoempresta.

3. Regras de negócio

Usuários e acesso

- **RN01.** O cadastro público cria sempre um MEMBRO. O papel nunca vem do cliente.
- **RN02.** O e-mail é único e normalizado (sem espaços, minúsculo).
- **RN03.** Usuário inativo não consegue fazer login.
- **RN04.** O ADMIN não pode alterar o próprio papel nem desativar a própria conta.
- **RN05.** O primeiro ADMIN é criado direto no banco (ou por migração).

Catálogo

- **RN06.** O ISBN é único.
- **RN07.** Todo livro tem uma categoria e pelo menos um autor.
- **RN08.** O nome da categoria é único, sem diferenciar maiúsculas.
- **RN09.** Não é possível excluir categoria, autor ou livro que tenha vínculos (409).
- **RN10.** O código do exemplar é único. Um exemplar com empréstimo (atual ou passado) não pode ser excluído.
- **RN11.** Um exemplar EMPRESTADO não pode ter o status alterado manualmente. Só DISPONIVEL, MANUTENCAO e PERDIDO podem ser definidos à mão.

Empréstimos

- **RN12.** Para emprestar, o usuário precisa estar ativo.

- **RN13.** Cada usuário pode ter no máximo **3 empréstimos ativos** (422).
- **RN14.** Usuário com **multa pendente** (multa maior que zero e não paga) não pode pegar emprestado (422).
- **RN15.** O pedido de empréstimo informa o **livro**, e o sistema escolhe um exemplar DISPONIVEL. Sem exemplar disponível, retorna 409.
- **RN16.** O prazo padrão é de **14 dias**.
- **RN17.** Ao emprestar, o exemplar passa para EMPRESTADO. Ao devolver, volta para DISPONIVEL.
- **RN18.** Devolução só vale para empréstimo ATIVO. A data de devolução é a data atual.
- **RN19.** A multa é **R$ 1,00 por dia de atraso** (dias entre a data prevista e a devolução). Sem atraso, a multa é zero.
- **RN20.** Renovação: só para empréstimo ATIVO, **não atrasado**, no máximo **1 vez**, e apenas se **não houver reserva ativa de outro usuário** para o livro. Soma 7 dias à data prevista atual.
- **RN21.** O pagamento da multa só é aceito se a multa for maior que zero e ainda não estiver paga.
- **RN22.** "Atrasado" **não é um status salvo**. É calculado: empréstimo ATIVO com data prevista anterior a hoje.
- **RN23.** Um membro só enxerga os **próprios** empréstimos e reservas. Para os dos outros, a resposta é 404.

Reservas

- **RN24.** Só é possível reservar um livro que **não tenha exemplar disponível** (409 caso contrário).
- **RN25.** Cada usuário tem no máximo uma reserva ATIVA por livro (409).
- **RN26.** Quando o usuário pega emprestado o livro que reservou, a reserva passa para ATENDIDA.
- **RN27.** O usuário só cancela as próprias reservas.

Convenção de erros

Status	Quando
400	Dados inválidos (validação)
401	Sem token, token inválido ou credenciais erradas
403	Logado, mas o papel não permite

Status	Quando
404	Recurso não existe (ou não é seu)
409	Conflito com o estado atual (ISBN repetido, sem exemplar, reserva duplicada)
422	Regra de negócio violada (limite de empréstimos, multa pendente, renovação não permitida)

Padronize as respostas de erro com `ProblemDetail`.

4. Arquitetura do repositório

Organização **por funcionalidade**, com as camadas dentro de cada pacote.

```
biblioteca/
├── pom.xml
├── mvnw / mvnw.cmd
├── Dockerfile
├── docker-compose.yml
├── .dockerignore
├── .gitignore
├── .env.example
├── README.md
├── docs/
│   ├── diagrama-classes-biblioteca.puml
│   └── diagrama-er-biblioteca.puml
└── src/
    ├── main/
    │   ├── java/com/seuprojeto/biblioteca/
    │   │   ├── BibliotecaApplication.java
    │   │   ├── config/        SecurityConfig, BibliotecaProperties,
ClockConfig, OpenApiConfig
    │   │   ├── auth/          AuthController, AuthService, JwtService,
JwtAuthenticationFilter,
    │   │   │                  UserDetailsServiceImpl, RegistroRequest,
LoginRequest, TokenResponse
    │   │   ├── usuario/       Usuario, Papel, UsuarioRepository,
UsuarioService,
    │   │   │                  UsuarioController, AdminUsuarioController,
DTOs
    │   │   ├── categoria/     Categoria, repository, service, controller,
DTOs
    │   │   ├── autor/         Autor, repository, service, controller, DTOs
    │   │   ├── livro/         Livro, repository, service, controller,
```

```
LivroFiltro, DTOs
    |   |   ├── exemplar/      Exemplar, StatusExemplar, repository, service,
controller, DTOs
    |   |   ├── emprestimo/    Emprestimo, StatusEmprestimo, repository,
service,
    |   |   |                  controller, EmprestimoFiltro, DTOs
    |   |   ├── reserva/       Reserva, StatusReserva, repository, service,
controller, DTOs
    |   |   └── exception/     RecursoNaoEncontradoException,
ConflitoException,
    |   |                      RegraDeNegocioException,
GlobalExceptionHandler
    |   └── resources/
    |       ├── application.yml
    |       └── db/migration/
    |           ├── V1__usuarios.sql
    |           ├── V2__catalogo.sql
    |           ├── V3__exemplares.sql
    |           ├── V4__emprestimos.sql
    |           └── V5__reservas.sql
    └── test/java/com/seuprojeto/biblioteca/
        ├── auth/          JwtServiceTest
        ├── emprestimo/    EmprestimoServiceTest (regras, com Clock fixo)
        ├── livro/         LivroServiceTest
        └── integracao/    testes de endpoints com banco real
(Testcontainers)
```

Regras de camada

- **Controller:** recebe, valida (`@Valid`) e repassa. Sem regra de negócio.
- **Service:** todas as regras e as transações (`@Transactional`).
- **Repository:** só acesso ao banco.
- **DTOs (records):** a API nunca expõe entidades.
- **Dependência entre features:** permitida de **service para repository** de outra feature. Evite service chamando controller, e evite dependências circulares.

5. Modelo de dados

Tabelas

usuario

Coluna	Tipo	Restrições
id	BIGSERIAL	PK
nome	VARCHAR(120)	NOT NULL
email	VARCHAR(180)	NOT NULL, UNIQUE
senha_hash	VARCHAR(100)	NOT NULL
papel	VARCHAR(20)	NOT NULL, DEFAULT 'MEMBRO', CHECK (MEMBRO, BIBLIOTECARIO, ADMIN)
ativo	BOOLEAN	NOT NULL, DEFAULT TRUE
criado_em	TIMESTAMPTZ	NOT NULL, DEFAULT now()

categoria

Coluna	Tipo	Restrições
id	BIGSERIAL	PK
nome	VARCHAR(80)	NOT NULL, UNIQUE
descricao	VARCHAR(255)	

autor

Coluna	Tipo	Restrições
id	BIGSERIAL	PK
nome	VARCHAR(150)	NOT NULL
nacionalidade	VARCHAR(80)	

livro

Coluna	Tipo	Restrições
id	BIGSERIAL	PK
titulo	VARCHAR(200)	NOT NULL
isbn	VARCHAR(17)	NOT NULL, UNIQUE
ano_publicacao	SMALLINT	CHECK (maior que 0)

Coluna	Tipo	Restrições
sinopse	TEXT	
categoria_id	BIGINT	NOT NULL, FK categoria, ON DELETE RESTRICT
criado_em, atualizado_em	TIMESTAMPTZ	NOT NULL

livro_autor (tabela de ligação)

Coluna	Tipo	Restrições
livro_id	BIGINT	PK composta, FK livro, ON DELETE CASCADE
autor_id	BIGINT	PK composta, FK autor, ON DELETE RESTRICT

exemplar

Coluna	Tipo	Restrições
id	BIGSERIAL	PK
livro_id	BIGINT	NOT NULL, FK livro, ON DELETE RESTRICT
codigo	VARCHAR(30)	NOT NULL, UNIQUE
status	VARCHAR(20)	NOT NULL, DEFAULT 'DISPONIVEL', CHECK (DISPONIVEL, EMPRESTADO, MANUTENCAO, PERDIDO)
criado_em	TIMESTAMPTZ	NOT NULL

emprestimo

Coluna	Tipo	Restrições
id	BIGSERIAL	PK
usuario_id	BIGINT	NOT NULL, FK usuario
exemplar_id	BIGINT	NOT NULL, FK exemplar
data_emprestimo	DATE	NOT NULL
data_prevista_devolucao	DATE	NOT NULL

Coluna	Tipo	Restrições
data_devolucao	DATE	nulo enquanto ativo
renovacoes	SMALLINT	NOT NULL, DEFAULT 0
multa	NUMERIC(10,2)	NOT NULL, DEFAULT 0, CHECK (maior ou igual a 0)
multa_paga	BOOLEAN	NOT NULL, DEFAULT FALSE
status	VARCHAR(20)	NOT NULL, DEFAULT 'ATIVO', CHECK (ATIVO, DEVOLVIDO)
criado_em	TIMESTAMPTZ	NOT NULL

reserva

Coluna	Tipo	Restrições
id	BIGSERIAL	PK
usuario_id	BIGINT	NOT NULL, FK usuario
livro_id	BIGINT	NOT NULL, FK livro
status	VARCHAR(20)	NOT NULL, DEFAULT 'ATIVA', CHECK (ATIVA, ATENDIDA, CANCELADA)
criado_em	TIMESTAMPTZ	NOT NULL

Índices

Índice	Para quê
`livro (categoria_id)`	Filtro por categoria
`lower(titulo)` em `livro`	Busca por título sem diferenciar maiúsculas
`exemplar (livro_id, status)`	Contar e achar exemplares disponíveis
`emprestimo (usuario_id, status)`	Contar empréstimos ativos do usuário
`emprestimo (status, data_prevista_devolucao)`	Listar atrasados
Único parcial em `emprestimo (exemplar_id)` onde `status = 'ATIVO'`	Garante no banco que um exemplar nunca tem dois empréstimos ativos

Índice	Para quê
Único parcial em `reserva (usuario_id, livro_id)` onde `status = 'ATIVA'`	Uma reserva ativa por usuário e livro

Os índices únicos parciais são uma **defesa no banco** para o caso de a regra falhar no código (por exemplo, por concorrência).

Relacionamentos

Relação	Cardinalidade	Como é implementada
Categoria → Livro	1 para N	`livro.categoria_id`
Livro ↔ Autor	N para N	Tabela `livro_autor`
Livro → Exemplar	1 para N	`exemplar.livro_id`
Usuário → Empréstimo	1 para N	`emprestimo.usuario_id`
Exemplar → Empréstimo	1 para N (no máximo 1 ativo)	`emprestimo.exemplar_id` + índice parcial
Usuário → Reserva	1 para N	`reserva.usuario_id`
Livro → Reserva	1 para N	`reserva.livro_id`

6. Endpoints

Papéis: **P** público, **M** membro (qualquer logado), **B** bibliotecário ou admin, **A** só admin.

Autenticação e perfil

Método	Rota	Acesso	Corpo	Respostas
POST	`/auth/registro`	P	nome, email, senha	201, 400, 409
POST	`/auth/login`	P	email, senha	200 (token, tipo, expiraEm), 400, 401
GET	`/usuarios/me`	M		200

Administração de usuários

Método	Rota	Acesso	Corpo	Respostas
GET	`/admin/usuarios?papel=&ativo=&page=&size=`	A		200 (paginado), 403
PATCH	`/admin/usuarios/{id}/papel`	A	papel	200, 404, 409
PATCH	`/admin/usuarios/{id}/ativo`	A	ativo	200, 404, 409

Categorias

Método	Rota	Acesso	Corpo	Respostas
GET	`/categorias`	P		200
GET	`/categorias/{id}`	P		200, 404
POST	`/categorias`	B	nome, descricao	201, 400, 409
PUT	`/categorias/{id}`	B	nome, descricao	200, 404, 409
DELETE	`/categorias/{id}`	B		204, 404, 409

Autores

Método	Rota	Acesso	Corpo	Respostas
GET	`/autores?nome=&page=&size=`	P		200 (paginado)
GET	`/autores/{id}`	P		200, 404
POST	`/autores`	B	nome, nacionalidade	201, 400
PUT	`/autores/{id}`	B	nome, nacionalidade	200, 404
DELETE	`/autores/{id}`	B		204, 404, 409

Livros

Método	Rota	Acesso	Corpo	Respostas
GET	`/livros?titulo=&autorId=&categoriaId=&disponivel=&page=&size=&sort=`	P		200 (paginado, versão resumida)
GET	`/livros/{id}`	P		200 (detalhe com autores, categoria e contagem de exemplares), 404
POST	`/livros`	B	titulo, isbn, anoPublicacao, sinopse, categoriaId, autoresIds	201, 400, 404, 409
PUT	`/livros/{id}`	B	igual ao POST	200, 400, 404, 409
DELETE	`/livros/{id}`	B		204, 404, 409

Exemplares

Método	Rota	Acesso	Corpo	Respostas
GET	`/livros/{livroId}/exemplares`	B		200, 404
POST	`/livros/{livroId}/exemplares`	B	codigo	201, 400, 404, 409
PATCH	`/exemplares/{id}/status`	B	status	200, 404, 409
DELETE	`/exemplares/{id}`	B		204, 404, 409

Empréstimos

Método	Rota	Acesso	Corpo	Respostas
POST	`/emprestimos`	B	usuarioId, livroId	201, 404, 409, 422
GET	`/emprestimos?status=&usuarioId=&atrasados=&page=&size=`	B		200 (paginado)

Método	Rota	Acesso	Corpo	Respostas
GET	`/emprestimos/meus?status=&page=&size=`	M		200 (paginado)
GET	`/emprestimos/{id}`	M (dono) ou B		200, 404
POST	`/emprestimos/{id}/devolucao`	B		200, 404, 422
POST	`/emprestimos/{id}/renovacao`	M (dono) ou B		200, 404, 422
POST	`/emprestimos/{id}/pagamento-multa`	B		200, 404, 422

A resposta de empréstimo traz um campo calculado `atrasado`.

Reservas

Método	Rota	Acesso	Corpo	Respostas
POST	`/reservas`	M	livroId	201, 404, 409
GET	`/reservas/minhas`	M		200
DELETE	`/reservas/{id}`	M (dono)		204, 404
GET	`/reservas?livroId=`	B		200

Relatórios (desafio extra)

Método	Rota	Acesso	Descrição
GET	`/relatorios/livros-mais-emprestados?limite=10`	B	Ranking com contagem (consulta com `GROUP BY`)
GET	`/relatorios/atrasados`	B	Empréstimos atrasados com dias de atraso e multa estimada

Regras de segurança por rota

- `GET` em `/livros/**`, `/autores/**` e `/categorias/**` é público. `POST`, `PUT` e `DELETE` nas mesmas rotas exigem B.
- `/admin/**` exige A.
- Todo o resto exige estar logado, e o dono do recurso é sempre validado no service.

- Use `RoleHierarchy` para que ADMIN herde BIBLIOTECARIO, e BIBLIOTECARIO herde MEMBRO, em vez de listar vários papéis em cada regra.

7. Diagrama de classes

O diagrama completo está em `diagrama-classes-biblioteca.puml`. Para visualizar, cole o conteúdo em plantuml.com ou use a extensão PlantUML no VS Code ou IntelliJ.

Como ler

- **Entidades** (`<<Entity>>`) espelham as tabelas.
- **Records** (`<<record>>`) são os DTOs de entrada e saída.
- Setas cheias (`-->`) significam "usa" (dependência de campo).
- Setas pontilhadas (`..>`) significam "gerencia", "recebe" ou "devolve".
- Em cada pacote, a cadeia é sempre **Controller → Service → Repository**.

Relacionamentos entre entidades

Entidade A	Relação	Entidade B	Observação
Livro	N para 1	Categoria	`@ManyToOne`, lado dono
Livro	N para N	Autor	`@ManyToMany` com `@JoinTable`, use `Set`
Exemplar	N para 1	Livro	Composição: sem livro não existe exemplar
Emprestimo	N para 1	Usuario	`@ManyToOne`
Emprestimo	N para 1	Exemplar	`@ManyToOne`
Reserva	N para 1	Usuario	`@ManyToOne`
Reserva	N para 1	Livro	`@ManyToOne`

Dependências importantes entre features

- `EmprestimoService` usa `ExemplarRepository`, `ReservaRepository`, `UsuarioService` e `BibliotecaProperties`.
- `LivroService` usa `CategoriaRepository`, `AutorRepository` e `ExemplarRepository` (para contar disponíveis).
- `CategoriaService` e `AutorService` usam `LivroRepository` (para impedir exclusão com vínculos).
- `ExemplarService` usa `EmprestimoRepository` (para impedir exclusão com histórico).

8. Roteiro por fases

Cada fase termina testável. Só avance quando os critérios estiverem marcados.

Fase 0: ambiente

Projeto Spring Boot, PostgreSQL no Compose, Flyway, Swagger.

- [ ] Aplicação sobe e conecta ao banco
- [ ] Swagger abre em `/swagger-ui/index.html`
- [ ] `.gitignore` correto e repositório no GitHub

Fase 1: usuários, login e papéis

Reaproveite o que você fez na API de tarefas: cadastro com BCrypt, login com JWT, filtro, `UserDetailsServiceImpl`. Novidades: o enum `Papel` com três valores, o campo `ativo`, `RoleHierarchy`, `/usuarios/me` e `/admin/usuarios`.

- [ ] Cadastro cria MEMBRO, mesmo se o JSON trouxer `papel`
- [ ] Login funciona, e usuário inativo é recusado
- [ ] MEMBRO recebe 403 em `/admin/**`
- [ ] Promover alguém tem efeito imediato (mesmo token)
- [ ] ADMIN não altera o próprio papel

Fase 2: catálogo

Categoria, autor e livro, com muitos-para-muitos, paginação e filtros.

- [ ] CRUD de categorias e autores
- [ ] Livro criado com categoria e lista de autores
- [ ] ISBN duplicado retorna 409
- [ ] `GET /livros` é público, com paginação, ordenação e filtros por título, autor e categoria
- [ ] Excluir categoria ou autor com livros retorna 409
- [ ] Listar 20 livros não dispara 20 consultas extras (verifique no log SQL)

Fase 3: exemplares

- [ ] Cadastro de exemplares por livro
- [ ] Filtro `disponivel=true` em `/livros`
- [ ] Contagem de exemplares totais e disponíveis no detalhe
- [ ] RN11 respeitada (EMPRESTADO não muda à mão)

Fase 4: empréstimos (o núcleo)

- [ ] Emprestar escolhe um exemplar disponível e o marca EMPRESTADO
- [ ] 4º empréstimo ativo do mesmo usuário retorna 422
- [ ] Livro sem exemplar disponível retorna 409
- [ ] Devolução libera o exemplar e calcula a multa corretamente
- [ ] Usuário com multa pendente é barrado, e volta a poder após o pagamento
- [ ] Renovação respeita todas as condições da RN20
- [ ] Membro só vê os próprios empréstimos
- [ ] Testes unitários das regras usando um `Clock` fixo

Fase 5: reservas e relatórios

- [ ] Reservar só quando não há exemplar disponível
- [ ] Reserva ativa de outro usuário bloqueia a renovação
- [ ] Emprestar o livro reservado marca a reserva como ATENDIDA
- [ ] Pelo menos um relatório

Fase 6: qualidade e entrega

- [ ] Testes de integração com Testcontainers cobrindo o fluxo emprestar → devolver
- [ ] Dockerfile e `docker compose up` subindo tudo
- [ ] README com como rodar, usuários de exemplo e lista de endpoints
- [ ] Segredos fora do código (variáveis de ambiente)

9. Configuração sugerida

Agrupe as regras numéricas em um bloco do `application.yml`, lido por uma classe tipada (`@ConfigurationProperties`):

Propriedade	Valor padrão
`biblioteca.prazo-dias`	14
`biblioteca.renovacao-dias`	7
`biblioteca.max-renovacoes`	1
`biblioteca.limite-emprestimos-ativos`	3
`biblioteca.multa-por-dia`	1.00

Assim, mudar uma regra não exige mexer no código.

10. Cenários de aceite (roteiro de testes manuais)

1. Cadastre `ana` (membro), `bia` (membro) e `carlos`. Promova o `carlos` a BIBLIOTECARIO pelo banco.
2. Visitante consegue listar livros, mas recebe 401 ao tentar criar um.
3. Ana (membro) recebe 403 ao criar um livro. Carlos consegue.
4. Carlos cadastra categoria, dois autores, um livro com os dois autores e **2 exemplares**.
5. Carlos empresta o livro à Ana (201). O livro mostra 1 exemplar disponível.
6. Carlos empresta à Bia. Agora há 0 disponíveis.
7. Um terceiro empréstimo do mesmo livro retorna 409.
8. Ana reserva o mesmo livro (409, porque ela já o tem?) ou outro usuário reserva (201). Defina e documente o comportamento.
9. Com um `Clock` ajustado para 20 dias à frente, devolva o empréstimo da Ana e confira multa de R$ 6,00 (6 dias de atraso).
10. Ana tenta pegar outro livro: 422 por multa pendente. Carlos registra o pagamento, e Ana consegue.
11. Ana tenta ver o empréstimo da Bia pelo id: 404.
12. Cadastre um usuário enviando `"papel": "ADMIN"` no JSON: o resultado deve ser MEMBRO.
13. Desative a Bia. O login dela passa a falhar.
14. Duas requisições simultâneas emprestando a **última** cópia: apenas uma deve vencer, e a outra recebe 409.

11. Armadilhas para prestar atenção

Sem entregar a solução, os pontos em que projetos assim costumam tropeçar:

- **Muitos-para-muitos:** use `Set`, defina o lado dono, e **não** coloque `CascadeType.REMOVE` em Livro → Autor (apagar um livro não deve apagar autores).
- **Problema N+1 na listagem:** buscar livros e depois autores e categoria um a um. Pesquise `@EntityGraph` e `JOIN FETCH`.
- **Paginação com coleções:** `JOIN FETCH` de uma coleção junto com `Pageable` gera um aviso do Hibernate (paginação em memória). Pense em duas consultas, ou em `@BatchSize`.

- **Concorrência na última cópia:** a verificação "tem exemplar disponível?" e a marcação "emprestado" não são atômicas. Há duas defesas: **bloqueio** da linha ao buscar o exemplar, e o **índice único parcial**, tratando a violação como 409.
- **Dinheiro:** use `BigDecimal` e `NUMERIC`, nunca `double`.
- **Datas:** `LocalDate` para datas sem hora (empréstimo e devolução), `Instant` para momentos (criação). Injete um `Clock` para testar atraso sem esperar dias.
- **Atraso calculado:** se você gravar "ATRASADO" no banco, precisará de uma rotina que o atualize todo dia. Calcular na hora é mais simples e sempre correto.
- **Filtros opcionais:** consultas com vários filtros que podem ser nulos pedem `Specification` ou uma `@Query` com condições opcionais.
- **Rotas públicas só para GET:** a configuração de segurança precisa casar **método HTTP + caminho**.
- **Import errado:** confira sempre que `Authentication` vem de `org.springframework.security.core`.
- **Migrações:** nunca edite uma migração já aplicada. Corrija com uma nova versão.

12. Desafios extras

- Endpoint `GET /usuarios/me/multas` com o total devido.
- Notificação de atraso (uma tarefa agendada com `@Scheduled` que apenas registra em log).
- Expiração automática de reservas antigas.
- Histórico de alterações de papel (tabela de auditoria).
- Cache da listagem de categorias.
- Refresh token ao lado do token de acesso.

13. Como pedir revisão

Quando terminar uma fase, mande o código dessa fase (as classes novas e as migrações) junto com o resultado dos testes ou o log do erro. Isso permite revisar o que está certo, o que falta e o que pode ser simplificado, sem reescrever o seu trabalho.